Architectural Justification

Academic Elective Bidding & Allocation System — Lab 3, Problem Statement #05

We chose Layered (3-Tier) Architecture for the Academic Elective Bidding & Allocation System.

Reason 1: Clear Separation of Concerns

The system cleanly separates the Client Layer (Student Portal, Registrar Console) from the Service Layer (Bidding Service, Validation Service, Allocation Engine) and the Data Layer (Database). This means prerequisite rules, timetable-collision logic, and the allocation algorithm can all change from semester to semester without touching the user interface, and the database schema can evolve without breaking the UI. Each layer only talks to the layer directly below it through a well-defined interface, which keeps the codebase easy to understand and maintain for a student information system that will be extended over many academic years.

Reason 2: Enforced, Consistent Data Access

Every request that needs data — submitting a bid, validating a prerequisite, or triggering the allocation solver — must pass through the Service Layer before it reaches the Database; no client component is allowed to call the Database directly. This guarantees a single, consistent path for reading and writing bids, capacities, and results, which is essential for the correctness of the credit-based allocation algorithm (FR-004): the Allocation Engine can trust that all bid data it reads has already passed through the Bidding Service's validation rules.

Security Advantage

Because the Database sits behind the Service Layer, the Registrar Console cannot write to the database directly — it must go through the Allocation Engine's AllocationControlInterface to trigger the solver, and that service enforces authentication and role checks before any data is touched. This prevents a compromised or buggy client from bypassing business rules and writing invalid capacity or allocation data straight into the database, which is exactly the kind of direct client-to-data access that a layered design is meant to prevent.

Performance Benefit

Because all data access is funnelled through a well-defined Service Layer, the Database layer can be optimized independently — for example, adding indexes or read replicas for the BidRecordInterface and CourseDataInterface calls that spike during the 5,000-bid peak bidding window — without requiring any change to the Bidding Service, Validation Service, or the client applications above it. This lets the system meet the sub-30-second allocation target in NFR-001 by tuning the data layer in isolation.